

Multiphase Transport Scheme for Hybrid Finite-Element Finite-Volume Meshes:

Refactoring of NCFVT Framework

Stephan K Matthäi, Philip Weiss, Thomas Driesner, Sebastian Geiger

CSMP Development Consortium

Ideas developed on the basis of joint meeting at HWU, May 2012

Goal: Transport of fluid phases with components

Composite Variables: that can be derived from ArrayVariable...

*Black Oil Model: Geiger et al.
09' (SPE107485)*

- Mass per unit volume vector: $\mathbf{m} = [M_o, M_g, M_w]^T$
- Concentrations of components, i (oil, gas, water) in phases, j (h=heavy, l=light, a=aqueous):
$$\mathbf{M} = \begin{pmatrix} M_{oh} & M_{ol} & 0 \\ M_{gh} & M_{gl} & M_{ga} \\ 0 & 0 & M_{wa} \end{pmatrix}$$
- Total component concentrations therefore are:
$$M_i = \sum_{j=h,l,a} M_{ij}, \quad i = o, g, w$$
- Partitioning of components among phases is described by:
$$\mathbf{R} = \begin{pmatrix} 1 & R_l & 0 \\ R_h & 1 & R_a \\ 0 & 0 & 1 \end{pmatrix}$$
$$R_h = \frac{M_{gh}}{M_{oh}}, \quad R_l = \frac{M_{ol}}{M_{gl}}, \quad R_a = \frac{M_{ga}}{M_{wa}}$$
- Solution ratios, R_x are used to quantify the partitioning:

Store matrices as new ARRAY variables?

Simplified transport equations

- Mass balances
(excluding sources and sinks)

$$\phi \frac{\partial M_i}{\partial t} + \nabla \cdot \mathbf{F}_i = 0, \quad i = o, g, w,$$

$$\phi \frac{\partial \mathbf{m}}{\partial t} + \nabla \cdot (\mathbf{R}\mathbf{B}^{-1}\mathbf{f}) = 0$$

- Component fluxes

$$\mathbf{F}_i = \sum_{j=h,l,a} \mathbf{v}_j \frac{M_{ij}}{u_j}$$

- Phase velocities

$$\mathbf{v}_j = -\lambda_j \left[\nabla (p + p_{cja}) - \rho_j \mathbf{g} \right]$$

- Where the mobilities are (but this is not mass-based?!)

$$\lambda_j = \mathbf{k} \frac{k_{rj}(s_j)}{\mu_j}, \text{ and } \lambda_t = \sum_{j=h,l,a} \lambda_j$$

- We also need sources and sinks and capillary / gravitational terms!

Isn't this only a slightly compressible formulation?

Hamdi and Vostok's approach (2012)

Compositional black-oil model

$$F_c = \frac{\partial}{\partial t} \left(\phi \sum_p x_{cp} \rho_p S_p \right) - \operatorname{div} \sum_p x_{cp} \rho_p \left(\mathbf{k} \frac{k_{rp}}{\mu_p} \nabla p_p \right) + \sum_p x_{cp} \rho_p q_p, \quad c = 1, \dots, n_c$$

Thermodynamic constraints

$$f_{ij}(p, T, x_{ij}) - f_{ik}(p, T, x_{ik}) = 0, \quad \forall j \neq k, \quad i = 1, \dots, n_c.$$

Different incarnations:

Black-oil based on mass fractions

Slightly-compressible using saturations

Objectives and prerequisites

Refactor- current transport scheme so that it supports more flexible calculations:

- Eliminate (Explicit)**StencilProcessor** and **NodeCenteredFiniteVolumeTransport** – based subclasses
- Make functionality of **FiniteVolumeAlgorithm** more transparent
- Support component transport in up to 3 phases
- Store / keep track of the transported components / composite variables (do not change in a transport step)
- FV centered saturation gradients using ported Karen's method
- Phase by phase formulation
- (Generic) mass-fraction based formulation for fully compressible flow
- (Specialisation) speed-optimised slightly- compressible scheme because it is faster as it does not require to computation of the composite variables
- For all components compute facet fluxes in one go

What we can achieve on this workshop

- Design generic transport scheme
- Implement explicit two-phase formulation for slightly compressible flow with gravity and capillary contributions as a first application
- Make a decision on how to calculate timestep size: CFL or other method to calculate timestep size

Support Wish List

- LocalVariableStorage: new interfaces AddTo(), MultiplyWith() to modify values without touching the variable flags?

Outline

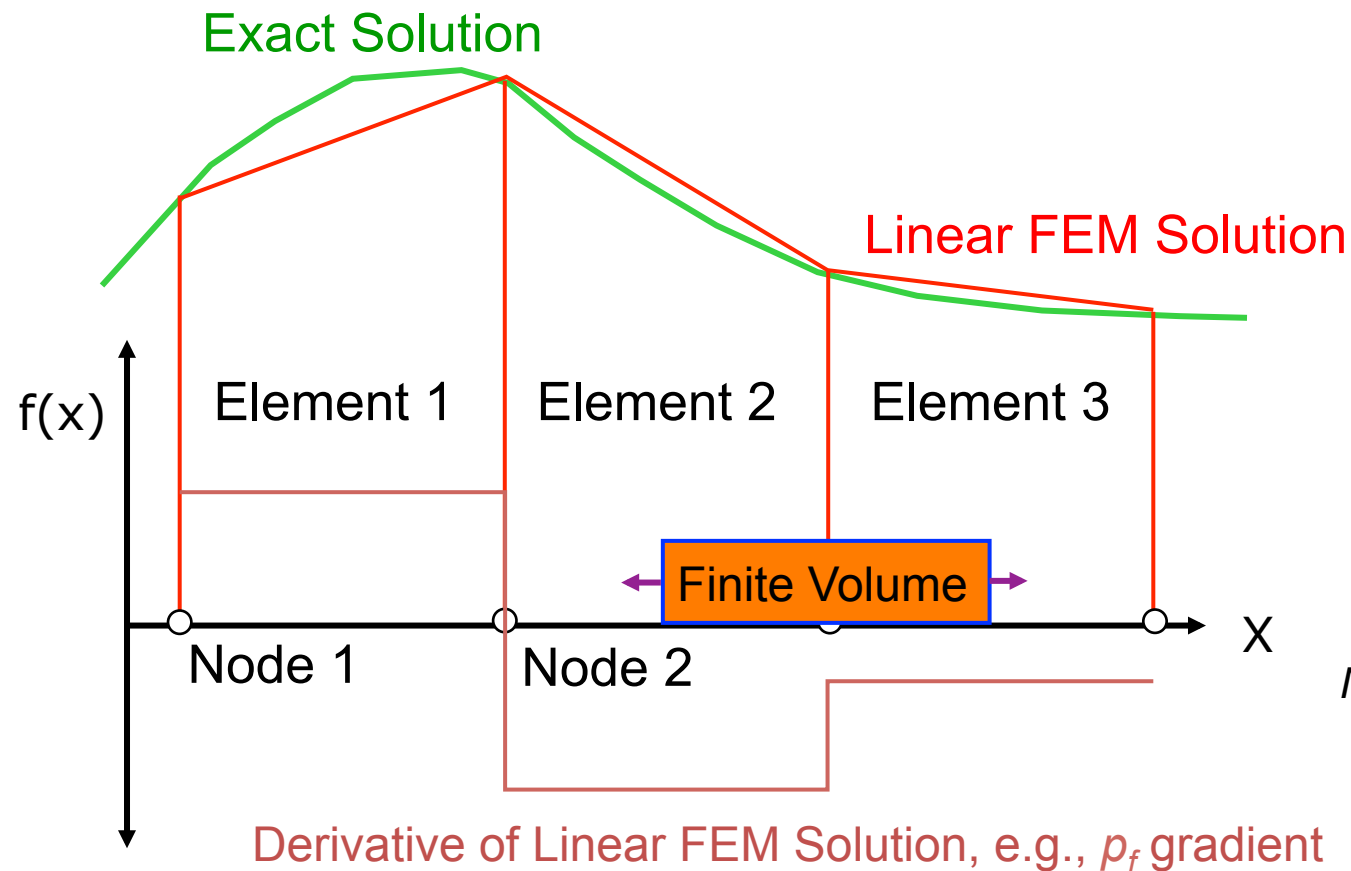
1. **What should be kept from our old scheme**
and what's new in CSMP++
2. Design specifications and considerations
3. Phase by phase transport
4. Design prototype: classes and use-case

FE-FVM Discretisation

Different requirements seem to exist here:
(*ETHZ* wants material properties on the nodes, *MUL* on the elements)

$$\text{FEM} \int_V \frac{\mathbf{k}}{\mu} \nabla p dV$$

$$\text{FV} \oint_{FV} \frac{k_n}{\mu} \frac{\partial p}{\partial \mathbf{n}} dS$$

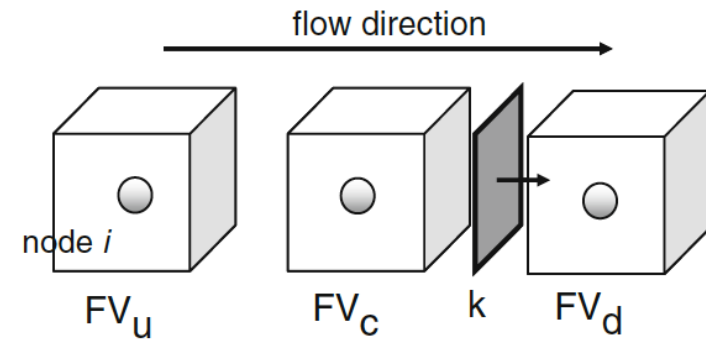
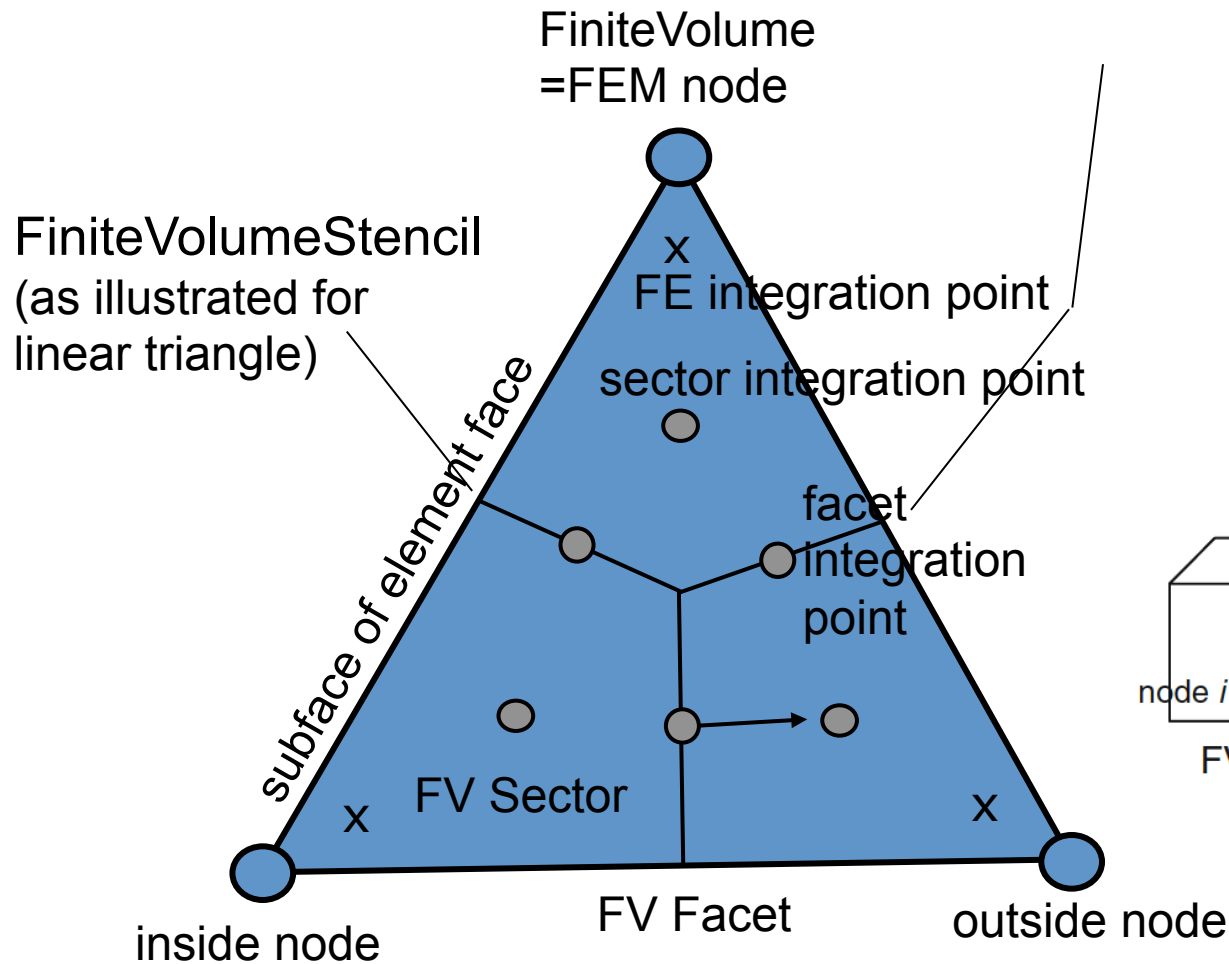


cf., Baliga & Patankar,
1980,
Numerical Heat Transfer,
Paluszny, Matthai and
Hohmeyer, 2007,
Geofluids

Basic terminology

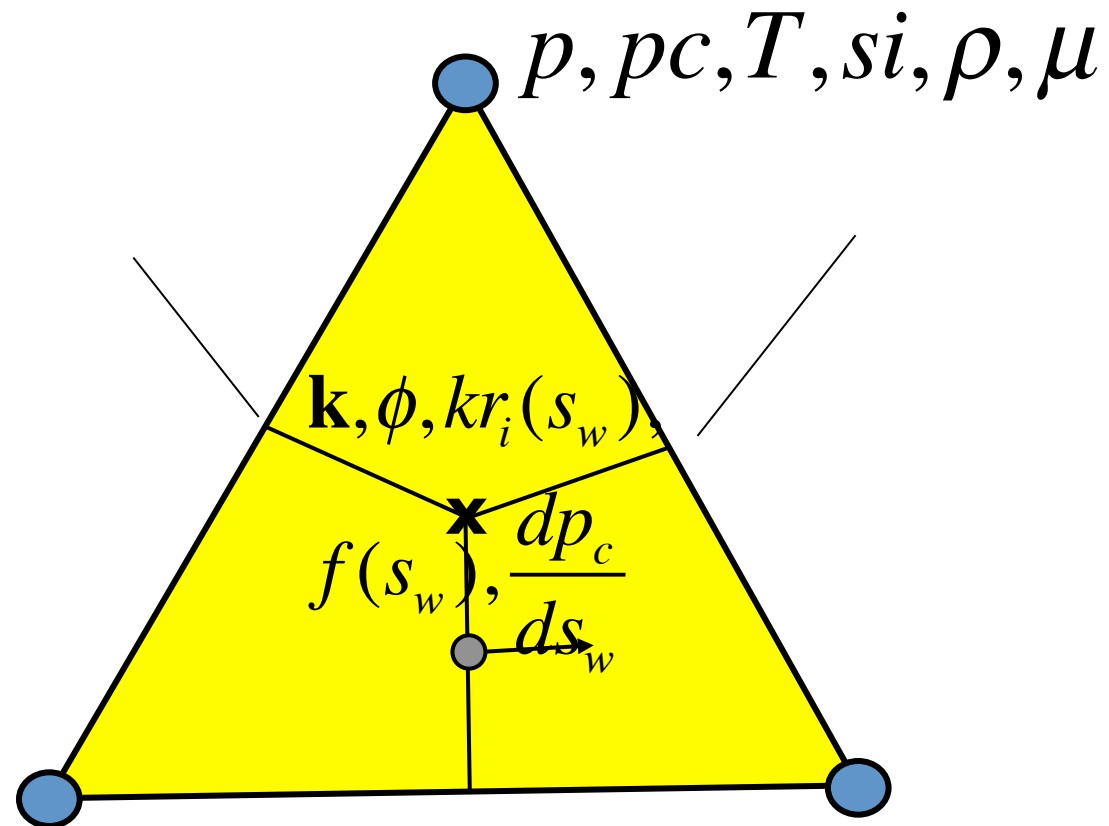
NEW!

```
enum PLACEMENT { ELEMENT_INTEGRATION_POINT = 1,
                  SECTOR_INTEGRATION_POINT = 2,
                  FACET_INTEGRATION_POINT = 3,
                  NODE,
                  EDGE,
                  FACE,
                  INTER_FACE,
                  ELEMENT,
                  REGION,
                  BOUNDARY,
                  SPLIT_BOUNDARY,
                  MODEL,
                  UNDEFINED };
```

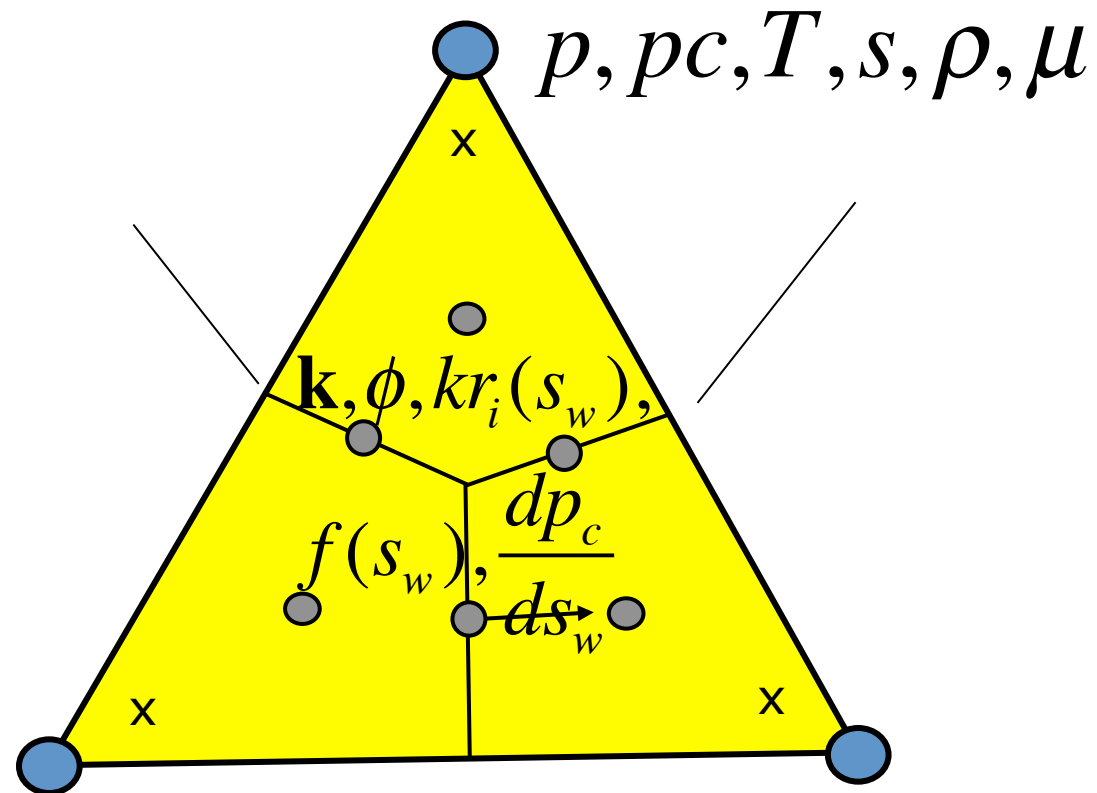


NB: Facet normals always point to node with higher local number

Variable placement (Barycentric formulation)

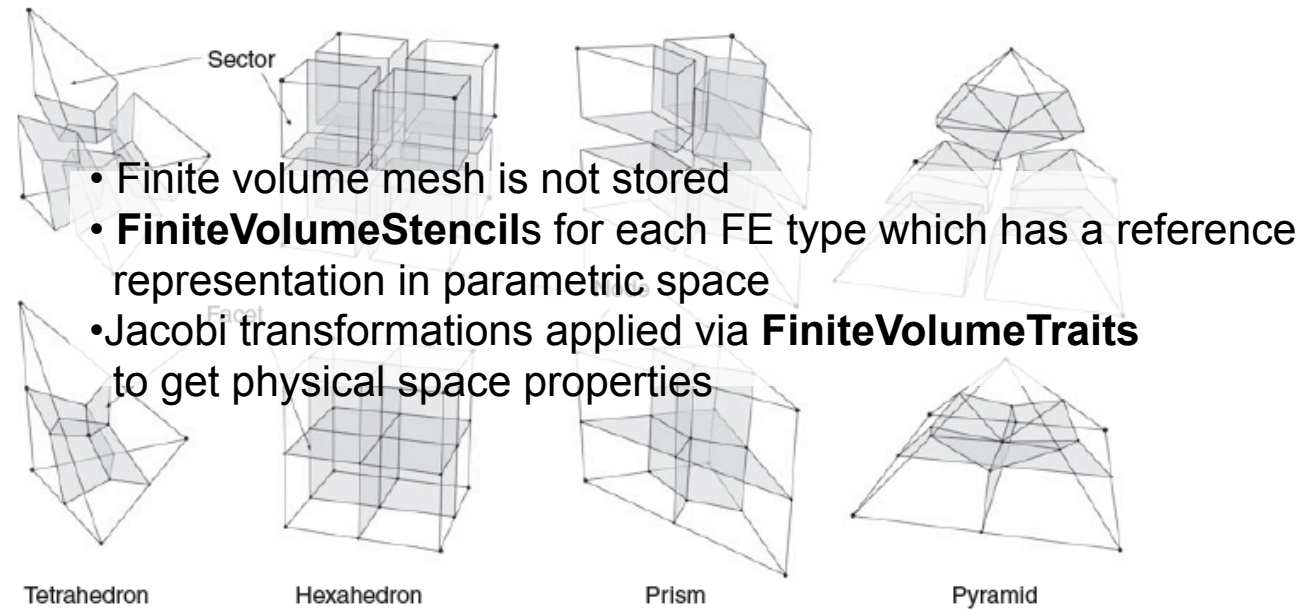


Variable placement (integration-point based formulation)



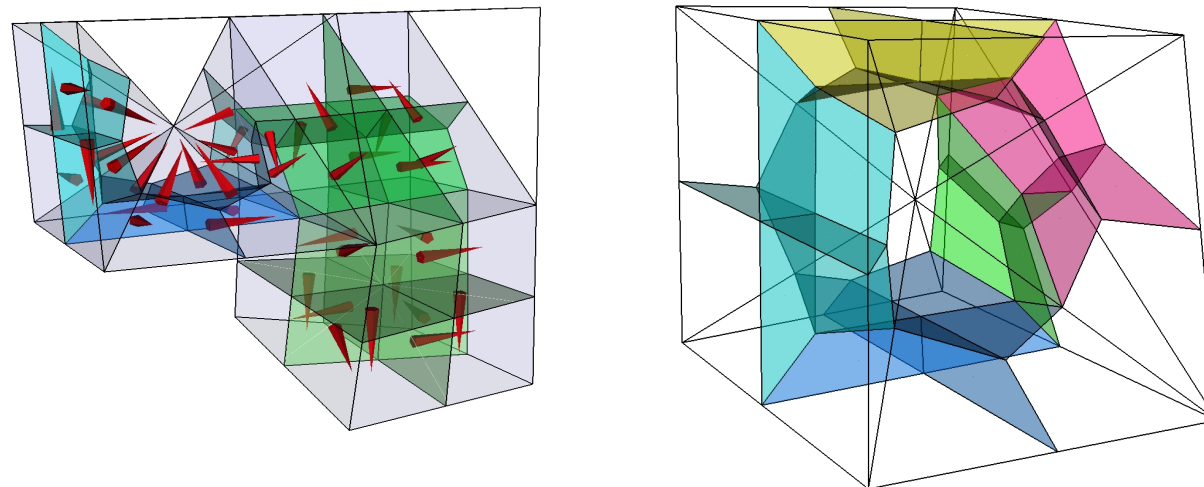
Finite-element finite-volume stencils

For
single
element
types



Paluszny et al. (2007, Geofluids)

FV cells
shared by
different
element
types



Access to FiniteVolume properties / parameters

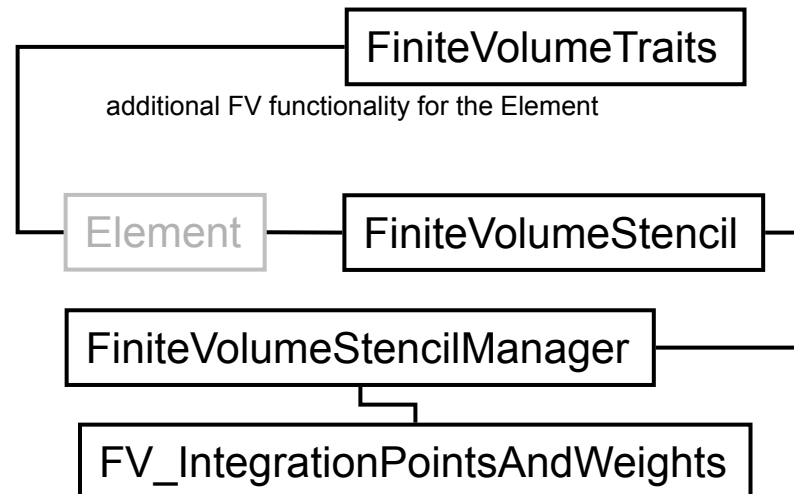
through **FiniteVolumeTraits** added to Element / Face / Interface: `e.SectorVolume(i);`

- As fast as possible, but it is better to store values if these are used repeatedly

```
double64 SectorVolume( size_t iSector ) const;
double64 FacetArea( size_t iFacet ) const;
Point<3U> FacetNormal( size_t iFacet ) const;
double64 FacetIntegral( size_t iFacet, const Index& ) const;
double64 SectorIntegral( size_t iSector, const Index& ) const;
double64 PropertyValueAtFacetIntegrationPoint( size_t iFacet, size_t, const Index& ) const;
double64 PropertyValueAtSectorIntegrationPoint( size_t iSector, size_t, const Index& ) const;
double64 ProjectionOnFacetNormal( size_t iFacet, const VectorVariable<3U>& ) const;
void N_AtFacetIntegrationPoint( size_t iFacet, size_t ip ) const;
void N_AtSectorIntegrationPoint( size_t iSector, size_t ip ) const;
void N_At( const Point<3U>& rst ) const;
Point<3U> RstToXYZ( const Point<3U>& rst ) const;
etc.
```

finiteVolumeAuxiliaryFunctions + FV interface of element, face or interface

*Framework
provided
through
Model*



*To learn how these
computations are done,
look at Paluszny et al.
(2007, Geofluids)*

*FV in parametric
space representation*

*Container of
available stencils*

*data container
for initialisation*

Scope for improvement: FiniteVolumeTraits

- Speed-up: It may be possible to carry out velocity computations entirely in parametric space, saving several Jacobian operations
- Confusing functions in interface of **FiniteVolumeTraits**:

```
double64    FacetAreaMapped( size_t iFacet ) const;  
Point<3U>    FacetNormalMapped( size_t iFacet ) const;  
double64          ParametricFacetArea( size_t iFacet ) const;  
const Point<3U>& ParametricFacetNormal( size_t iFacet ) const;
```

- Is there an issue with velocity computations on polyhedral (multi-element type) meshes, where there appears to be a divergence in the velocity field?
- Option PYRAMID_WITH_TRIANGULAR_FACETS should be used always and flag should be removed

Keeping support functionality from **NodeCenteredFiniteVolumeTransport** ?

Class needs a rebuild, but SKM suggests that certain functions are kept:

- Alternative: Keep these functions but dismantle class?
 - At least the constructor / initialisation function are always needed
 - A transport class / visitor is also needed to manage accumulations / integrations etc.
 - CFL calculations etc. could be moved to **finiteVolumeAuxiliaryFunctions** ?
 - Complicated procedure to compute nodal volume fluid source terms for PointSourceRHS in FEM scheme should be replaced by pde_operators.
 - Transport variables not stored in NCFVT / FV_Parameter array, but on model
 - Implication: transport variables exist everywhere!
 - Single phase transport should also be associated with a phase?

What currently happens when the NCFV transport is instantiated

- Finite volume stencil class instances of **FiniteVolumeStencil** are constructed and initialized with data from **FV_IntegrationPointAndWeights** “singleton” for the different finite element types
- Specific stencils are associated with corresponding elements as managed by the **FiniteVolumeStencilManager**
- Storage for SECTOR INTEGRATION POINT and FACET_INTEGRATION_POINT variables is created
- **FV_Parameter** instances are initialized doing computations with the **FiniteVolumeStencil** objects and are stored in the STENCIL_DATA vector inside of the **NodeCenteredFiniteVolumeTransport**. Initialized are with facet normals, facet areas, and sector volumes for accelerated lookup and projections during run.
 - *NB: FV_Parameter data can now be stored on the mesh*
- FV-pore volumes and flux balance arrays are computed. If second order is required, a node connectivity graph and SMINMAX array values are created.

What currently happens in an 2nd order explicit transport calculation?

1. For the transport region, calculate CFL (/ time-stepping criterion)

- Element-by-element: from phase velocity based on combined viscous-gravitational-capillary phase velocity, but not sources
- Decision needed: compute only once using shock speed, vs. recompute for each transport increment

2. Compute FV centered saturations gradients using Karen's method

3. For (second order in space), find smin / smax upstream of each FV facet

MinMaxAdvectedProperty(); using node-to-node connectivity

3. Loop over all elements of the transport region to accumulate fluxes / sources

- stencil_.AccumulateExplicitAdvectionSolution2(this->SMINMAX, (*fvt), *(*eit), RESULT);
- compute limiter values

4. Accumulate boundary terms, compensating for inflow and outflow

AssignFluxBoundaryConditions();

5. Compose solution $S^{t+1} = S^t - dt/(\phi V_i) * \sum_j^{faces} A_j n \cdot [f_{vt}] + src$

6. Save computed saturations to FV

OutputResults(this->pref_, output_result_range);

CFL calculation

- CFL is currently calculated using the element length in the direction of flow, see NCFVT::**AnisotropicCourantIncrement()**. This only considers viscous and gravitationally driven flow since capillary spreading is computed with the FE method.
- A limiting value is specified by the user for the case when there is no flow.
- For explicit calculations, df/ds_{max} is used in the calculation of the 2-phase CFL limit because the resulting step size will be sufficiently small even when df/ds increases from the current to subsequent iterations. For implicit calculations this is not necessary and the virtual CFL method can be less restrictive.
- For explicit schemes, the CFL calculation must simultaneously address viscous, gravitational, and capillary terms $CFL = \max(\text{visc_CFL} \dots)$
- Even if one of these is zero a meaningful value needs to be established.
- Should the CFL method be virtual so that they can be overridden?

Constitutive relationships

- Are currently recovered from subclasses of the **TwoPhaseModel** (here we need extensions for 3-phase and perhaps for compressible flow)
- Idea (for element-based material properties): initialize 2-phase model for element (or specific internal site (=node etc.)), then compute terms accessible through interface.
- Current interface is a mess! – somebody has played with the base class!

```
void SaturationWettingPhase( double64 s_wetting );
void Permeability( double64 permeability );
double64 ViscosityNonWettingPhase() const;
double64 ViscosityWettingPhase() const;
double64 DensityNonWettingPhase() const;
double64 DensityWettingPhase() const;
double64 Saturation( size_t phase ) const;
double64 MobilityPhase( size_t phase ) const; // kr_i/mu_i
double64 TotalMobility() const;
double64 TotalMobilityMultiplier() const;
double64 DiffusionMultiplier( size_t phase ) const;
double64 CapillaryDiffusionMultiplier( size_t phase ) const;
double64 GravityMultiplier_G() const;
double64 GravityMultiplier_dGds() const;
double64 AdvectionMultiplier() const;
virtual double64 ShockSpeed() const;
virtual double64 ShockHeight() const;
```

```
virtual double64 EffectiveSaturation() const;
```

```
virtual double64 krn_Phase() const = 0;
virtual double64 krw_Phase() const = 0;
```

```
/// fractional flow
double64 f_Phase( size_t phase ) const;
```

```
/// derivative of fractional flow (used in advection multiplier); note that fw+fn=1, dfwdsw=dfndsn
virtual double64 dfds() const = 0;
```

```
/// maximum value of previous derivative
virtual double64 MaxFractionalFlowDerivative() const = 0;
```

```
/// G multiplier
double64 G() const;
```

```
/// derivatives of gravitational flow (advection multipliers)
virtual double64 dGds() const = 0;
```

```
virtual double64 pc_Phase( size_t phase ) const = 0;
```

```
/// capillary pressure derivatives (treat seff as for previous function)
virtual double64 dpds_Phase( size_t phase ) const = 0;
```

```
/// maximum value of dpds
```

What methods do we really need?

Outline

1. What should be kept from old scheme
and what's new in CSMP++
- 2. Design specifications and considerations**
3. Phase by phase transport
4. Design prototype: classes and use-case

Decision needed:

Placement of material properties

Element

- FE mesh can be used to represent material boundaries
- Saturation at material interfaces makes no physical sense (discontinuities can only be represented with Split Boundaries or FECFVM scheme)
- For element-centered saturation, upwind direction does not matter

Node

- Material boundaries are blurred and domain volumes cannot be estimated accurately
- Saturation is physically meaningful even where sharp property gradients exist
- Transmissibility multipliers are non-physical if different flow laws exist in different domains

Fractured reservoir simulation needs element-based material properties

Element-centered variable scheme: needs

To store at different time levels (in one step or by repeated accumulation):

- **finite volumes:**
 - Integrated pore volume -> SCALAR
 - Divergence of total velocity -> SCALAR
 - Sources or sinks -> SCALAR
 - Saturation -> ARRAY
 - Saturation gradient (for higher order scheme) -> VECTOR
 - Inflow versus outflow in the presence of sources and sinks -> SCALAR
- **facets:**
 - combined phase and components fluxes
 - phase velocities
- **elements:**
 - total velocity -> VECTOR
 - diffusivities
 - sector porosity and pore volumes
 - sector source terms (for FEM equations)
 - constitutive relationships

New goal: phase-by-phase component transport scheme with support for component exchange among phases

(Transfer among phases is accomplished by EOS classes; diffusion terms are computed by finite element operators accumulated into same solution matrix as terms from implicit version of transport scheme)

- Computations needed (first and second-order in space):
 - CFL time increment or else where applicable; else saturation change or inflow vs. outflow (for instance, when source terms dominate)
 - upwind direction (combined: capillary+diffusion+transport)
 - upwind saturation range
 - advection terms
 - node-centered saturation gradients (or bijective mapping (Matthai et al. 2009, TIPM; check!))
 - source terms (upwinded or not)
 - storage terms
 - limiter(s)

Implemented vs. conceptual functionality

- Calculating and updating porosity and pore volume (by Model?)
- Calculating and updating facet area and normal (by Model?)
- Read / store / range check / projections of variables
- Determine upwind nodes
- Calculate CFL or any other special requirements on time-step size
- Mass balance
- Scaling n components and m phases
- Calculating components
- Determine and store facet fluxes
- Accumulation of fluxes
- Time-discretization (implicit, explicit, first/second order)
- Iterations (e.g. Newton)
- Higher-order in space
- Limiting
- Boundary conditions
- Additional physics/chemistry
- Access and configuration of the SAMG_Solver from the outside

Efficiency considerations

- Whatever we do here has a great impact on speed!
- To avoid iffy code, interior domains should be visited first, then their boundaries
- Formulation should be suitable for DES implementation
- FV visitation mode has a great impact on speed: element-by-element means that facets are only visited once, but may not be possible in higher-order scheme when node-centered saturation gradients are used in the flux approximation
- CFL computations based on local shock speed (=element property), only work if the gravitational component of flow is small because shock speed is different in up- or down-dip flow

Outline

1. What should be kept from old scheme
and what's new in CSMP++
2. Design specifications and considerations
3. Phase by phase transport
4. **Design prototype: classes and use-case**

(New) classes?

FiniteVolumeManager

SaturationFunctions

- relperms, etc.
- mobilities

Fluid Properties

GenericGradientCalculator

TimeIncrement class

- Calculate CFL or other criteria
- Mass Balance
- Volume Based
- Time-discretization (implicit, explicit, first/second order)

*To be carried out and managed by **Model***

- If geometry changes, update porosity, pore volume, facet area and facet normal

EOS

Additional physics/chemistry

InteriorProcessor (base class)

Accumulation (LHS?)

BoundaryProcessor (base class)

Accumulation (RHS?)

FacetFlux

- Calculating components
- Determine and store facet fluxes

Explicit

- Accumulation of fluxes to vector
- Accumulation of sources to vector
- Add fluxes and sources
- Higher-order in space
- Limiting

Implicit (RHS vs. LHS accumulations)

- Accumulation order
- Accumulation of fluxes to matrix
- Accumulation of sources to matrix
- Iterations (e.g. Newton)
- Solver / SAMG

Classes – Colleoli Workshop

- SaturationFunction – relative permeability and capillary pressure
- Fluid – fluid properties, speciation, phases, components
- GenericPropertyGradient – FV centered gradients
- FlowFunctions – fractional flows, derivatives etc.
- FacetFlux – upwind directions and facet fluxes computed stencil by stencil
- FluxEvaluator – flux computations for entire region of computation
- Accumulator – assembling LHS and RHS operators
- VectorOperator – base class for RHS integrals and BCs
- MatrixOperator – base class for LHS integrals
- ExplicitTransport – assembly of explicit scheme solution
- ImplicitTransport – Newton-Raphson – LineSearch accumulation

FluidProperties

- Input are P , T , X
- (Maximum) number of phases
- Number of components per phase
- Mass fractions
- Phase densities
- Phase viscosities
- Saturations (in the case of mass-fraction based scheme)
- Component exchange among phases

SaturationFunction

- From saturation(s) provided externally computes $k_{ri}(sw)$, $p_c(sw)$, dp_c/ds
- Has access to the element interface (and initializes itself there)
- relative permeability can be a function of all kinds of parameters (hysteresis, velocity dependence, etc.)
- capillary pressure function

FlowFunctions

- Computes total mobility, fractional flows, derivatives, pc-derivatives etc. from input variables computed by **SaturationFunctions** and **FluidProperties**
- Default: all derivatives are evaluated numerically

FacetFluxCalculator

- To compute first- and second order facet fluxes
- Operations:
 - Compute facet normal permeabilities in the case of permeability tensors
 - Uses **FlowFunctions** to compute fractional flow, derivatives etc.
 - computes total mobility and velocity if needs
 - find *phase velocit(ies)* (viscous, gravitational and capillary) at the barycenter to determine the upwind directions for all facets
 - interpolate saturation using GenericGradientCalculator to find facet saturation
 - evaluate relperms (using all kinds of parameters) via SaturationFunctions
 - Get fluid properties (densitie(s), viscosity(s), concentrations etc.)
 - Evaluate mobilities and fractional flows
 - Compute facet fluxes
 - Store facet fluxes
 - NO LIMITING

The classes that carry out the transport

Combines accumulation process with machinery required to compute solutions / time-integrate transport equations.

ExplicitTransport

- In the simplest case, just writes the solution vector

ImplicitTransport

- Combines accumulation into SparseMatrix with solution with solver, range checks, etc.
- Used to implement iterative solution schemes, for instance, Newton Raphson with line search a la Christine
- Manages csmp::Solver class

LHS_Accumulator and RHS_Accumulator

- Provides PDE_Integrator style functionality to assemble flux / source / sink etc. terms into a linear algebra $Ax=b$ or simple vector framework so that a solution can be formed or computed.
- Supports the assembly of expressions involving operator
- Matrix accumulation by **MatrixOperator** and **VectorOperator** with state variables that specify how and when they should be accumulated.
- We need telling names for:
 - Pore volumes divided by time_increment into diagonal of matrix
 - Incoming (higher-order) fluxes coupling diagonal with off-diagonal
 - Outgoing(higher-order) fluxes added to diagonal
 - Sources / sinks for LHS and RHS
 - All these for RHS
 - Jacobians for iterative solutions
 - Diffusion type FEM integral operators
 - Any of these at different time-levels
 - Theta-weighting for second-order in time method
- Static polymorphism for speed

Operator options

- Divide / multiply-with time-increment
- Add, Subtract, Multiply
- LateAccumulate
- Lumping
- Theta-weighting

FluxEvaluator (for the entire region)

- Updates the fluid properties (including the saturations)
- If a second-order is requested, computes the finite-volume centered property gradients
- Calls the FacetFluxCalculator to compute facet fluxes of phases
- Integrates total incoming versus outgoing fluxes for each finite volume

Question: Limiting should be done on the phase flux (total). What does this imply for where the limiter is called (by FacetFluxEvaluator or decoupled)

TimeStepEvaluator

- Computes a time increment such that the amount of fluid that leaves any FV is not greater than what was in there
- For implicit schemes any relaxation is applied to this

ExplicitTransport and ImplicitTransport

- Drives
 - FluxEvaluator
 - TimeStepEvaluator
 - Accumulator
 - Solver
- Solution process can involve NewtonRaphson and LineSearch algorithms

A typical calculation?

1. Compute pressure
2. Compute total velocity etc.
3. **Update multiphase flow properties, phase densities and viscosities**
4. **Determine initial time increment**
5. **Accumulate (higher-order) interior stencils**
 - **Compute viscous, gravitational and capillary fluxes**
 - **Determine upwind direction on basis of composite flux**
 - **Establish flux balances and pressure-change dependent source / sink terms**
6. **Accumulate first order boundary stencils**
7. **Solve transport problem**
8. **Transfer results to model**